

PRÁCTICA 6

CONTROL DE MOTORES CON EL MCPWM

Juan Pedro Bandera Rubio (jpbandera@uma.es)

Como indicamos en la práctica anterior, LED Control puede emplearse para controlar servomotores de forma sencilla. Sin embargo, para controlar realmente bien un motor con el ESP32-S3 lo adecuado es usar el periférico MCPWM (Motor Control PWM). Este es el periférico que debes emplear para controlar los motores del microbot. Recuerda que tienes la documentación oficial sobre MCPWM aquí: <https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32s3/api-reference/peripherals/mcpwm.html>

El objetivo de esta práctica guiada es dar una visión de los fundamentos del control de motores con el ESP32-S3, y un ejemplo práctico de programación aplicable directamente a tu microbot. No se entra en detalles ni explicaciones profundas (que estaría bien afrontar, pero realmente caerían fuera del ámbito de la asignatura).

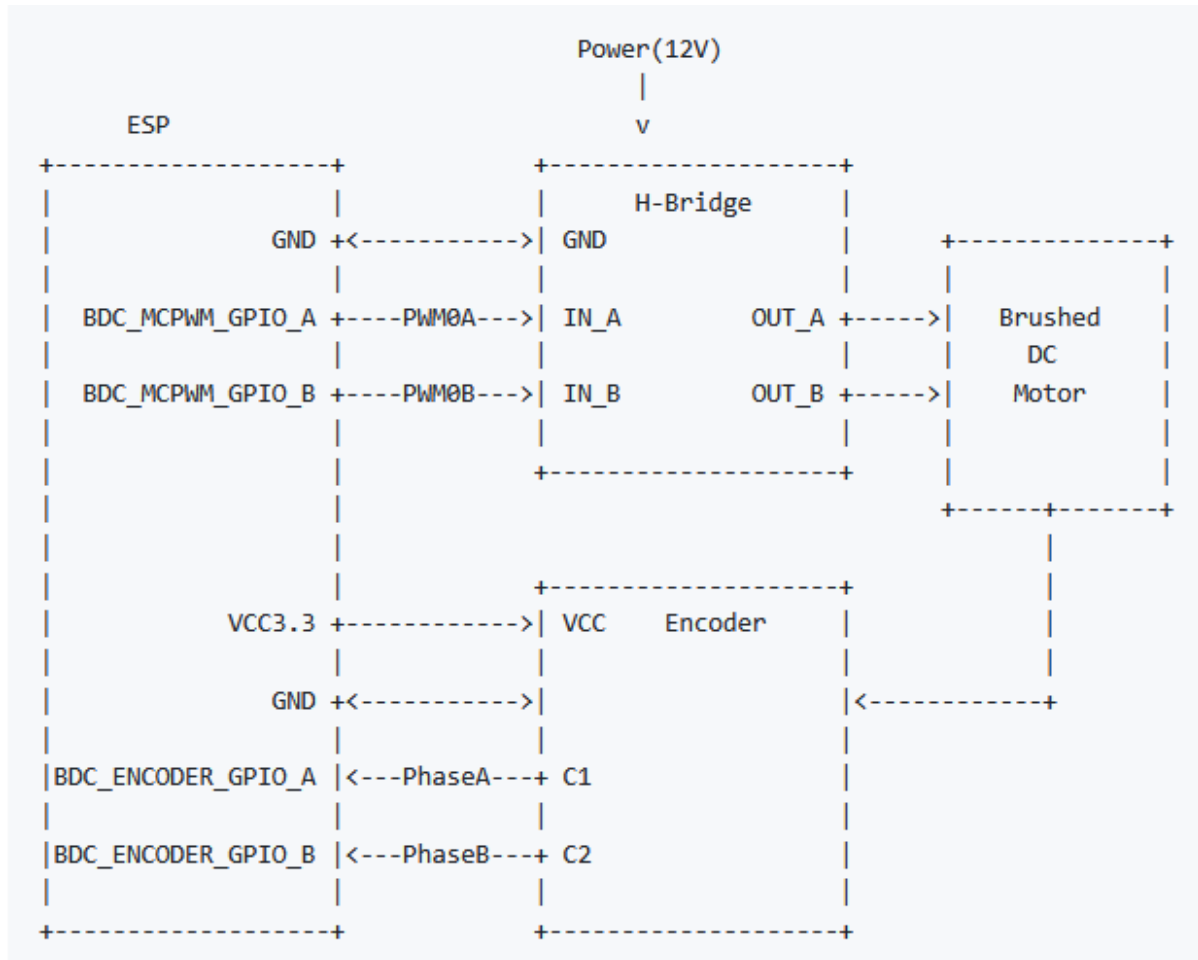
Para esta práctica, se proporciona el código de ejemplo *ESP32_S3_esp-idf-MCPWM*, que en este caso es una adaptación directa, con modificaciones mínimas, del código de ejemplo para control de motores Brushed DC (BDC) suministrado por Espressif para el ESP-IDF. Puedes acceder a ese código desde la documentación oficial o directamente en este enlace: https://github.com/espressif/esp-idf/tree/v6.0.2/examples/peripherals/mcpwm/mcpwm_bdc_speed_control

QUÉ QUIERES HACER: CONTROL PID DE UN MOTOR BDC

1. Quieres controlar un motor BDC, y para eso usarás dos señales PWM específicas. Estas señales las genera el MCPWM del ESP32-S3, pero no tienen suficiente potencia para controlar directamente el motor, así que las conectarás a un driver (puente en H) para poder **mover el motor**.
2. Si se tratase sólo de mover el motor al tuntún, con eso valdría. Pero quieres moverlo con precisión, controlar la velocidad, y los cambios de velocidad. Por eso necesitas saber a cuántas revoluciones gira el motor, y en qué sentido. Para eso, tendrás que **leer los valores de los encoders del motor BDC**.

- Finalmente emplearás un controlador **PID**, ejecutado periódicamente, para **controlar el movimiento del motor**. El PID, a partir de los valores leídos de los encoders, irá calculando y enviando los valores de velocidad adecuados al motor BDC.

El diagrama de conexión del código de ejemplo del ESP-IDF muestra bastante bien el proceso:



MOTORES BRUSHED DC (BLDC o BDC)

Este tipo de motores llevan usándose desde hace décadas (va para 100 años ya). Si tienes curiosidad sobre cómo funcionan, encontrarás muchísima información publicada al respecto.

En esta asignatura te proporcionaremos algunos detalles sobre su funcionamiento en la sesión de motores, más adelante. En este momento, para entender esta práctica, vamos a considerar el motor una caja negra, de la que nos interesa lo siguiente:

- Los motores BDC no son los más fuertes, pero proporcionan un control preciso.
- Tenemos que conectarnos a él a través de un puente en H para poder moverlo.

- Se mueve mediante unas señales PWM específicas generadas desde el periférico MCPWM. No te preocupes, tenemos una librería que nos abstrae de la construcción a bajo nivel de esas señales PWM. Bastará con indicar la velocidad a la que se quiere mover el motor.
- Proporciona velocidad y sentido de giro mediante dos señales de encoders.

En el microbot, los motores que se usan son los JGA25-371 DC. Tienes su datasheet en la pestaña de recursos de la asignatura. Como puedes ver allí, se alimenta a 12v y sus encoders tienen 12 pasos por revolución.

PUENTE EN H

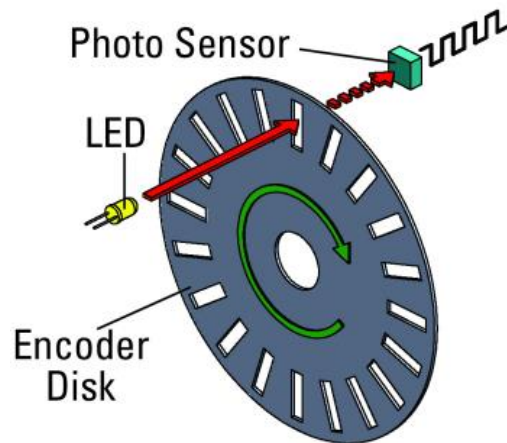
Es básicamente un driver, que permite proporcionar unas señales de salida de más potencia, a partir de unas entradas débiles. En este caso es necesario porque las salidas del ESP32-S3 no serían capaces de hacer girar los motores. Al igual que hemos hecho con el motor, en esta práctica consideramos el puente en H una caja negra.

En el microbot, se utilizan los DRV8871. Tienes el datasheet en la pestaña de recursos.

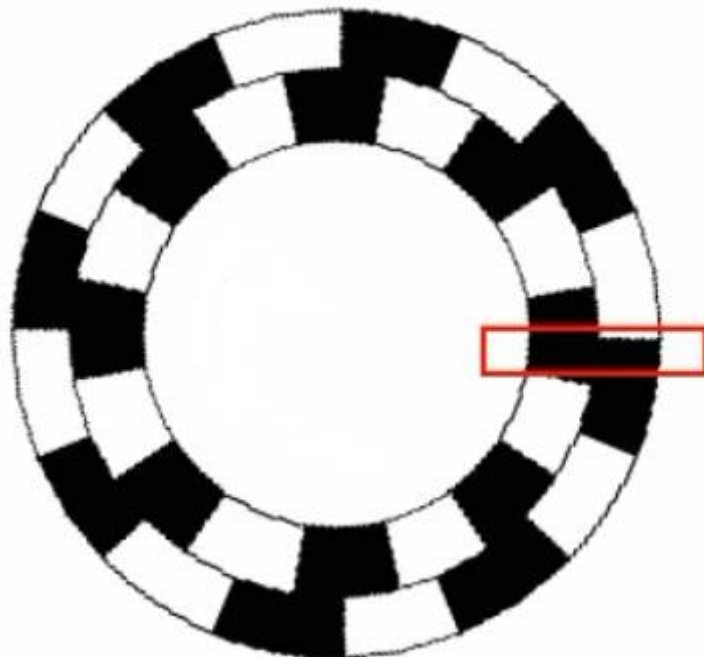
ENCODERS

Un encoder es básicamente un sistema que detecta los giros del motor. Son necesarios para conocer, por ejemplo, la posición en la que se coloca un servomotor (como los que controlan los alerones de un avión). También para conocer la velocidad de giro, y el sentido en que está girando el motor.

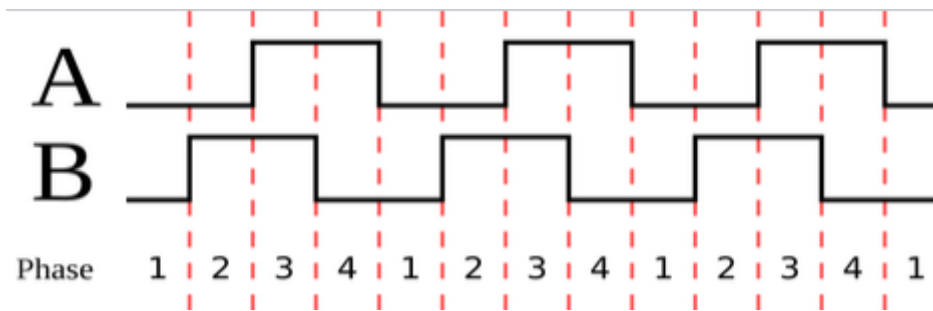
Los hay de diversos tipos, para entender su funcionamiento vamos a presentar aquí un encoder óptico, que se implementa con una rueda con segmentos negros/blancos o transparentes/opacos, y un sensor óptico que detecta los cambios conforme la rueda gira, conforme se muestra en la siguiente figura. Esto permite calcular la velocidad de giro a partir de la información proporcionada por un único sensor.



Sin embargo, en nuestro caso necesitamos conocer la **velocidad** y el **sentido** en que el motor gira. Con una sola señal no basta en este caso, necesitamos emplear dos sensores. Cada uno de ellos irá leyendo transiciones en dos ruedas, en las que estas transiciones están desfasadas 90 grados, como se muestra en la figura (donde un sensor lee la rueda interior, y otro la exterior):



Las señales recibidas por los sensores A y B tendrán este aspecto:



Y conforme la rueda gira, puedes conocer el sentido de giro viendo en qué orden van variando las señales A y B. Este GIF muestra bastante bien ese proceso:

https://mischianti.org/wp-content/uploads/2020/03/Incremental_directional_encoder.gif

Por ejemplo, si ambos sensores leen '0' (ambos en sector negro), y el sensor A, que lee la rueda interior, es el primero que cambia a '1' (blanco), entonces es que el motor gira en sentido antihorario. Sin embargo, si partiendo de ambos sensores en '0' es el sensor B el primero que cambia a '1', entonces es que el motor gira en sentido horario.

Este tipo de encoders son llamados **encoder de cuadratura**, y son los que usarás para detectar la velocidad y sentido de giro de los motores de tu robot.

CONTROL PID

Una vez se tiene la información del encoder, se sabe cómo se está moviendo el motor. A partir de ahí, es posible usar esa información para ir regulando la velocidad y sentido de giro del motor según como quieras que el robot se mueva.

Para tí, lo ideal será enviarle al robot órdenes de movimiento sin más ('avanza a 0.2 metros por segundo'). Y eso será lo que acabarás haciendo. Pero por debajo de eso, será necesario un controlador de más bajo nivel que traduzca esas órdenes en una serie de transiciones de movimiento que permitan al robot moverse de forma adecuada. Aquí, *adecuada* depende de tí. Por ejemplo, puede que quieras arrancar despacito y coger velocidad para evitar meneos bruscos, y frenar poco a poco para acabar parando justo donde querías. O quizá prefieras arrancar a toda velocidad, asumiendo que habrá vibraciones, y frenar de golpe, para llegar antes, asumiendo que puede que derrapes un poco. Es cuestión de cómo quieres que se comporte el robot.

Esta regulación del comportamiento de los motores se consigue tradicionalmente mediante el uso de controladores PID (Proporcional - Integral - Derivativo). Los usarás en esta asignatura, y en la sesión de motores se darán más explicaciones acerca de

ellos. Por ahora, para esta práctica, sólo es necesario que sepas que el controlador PID se configura con una serie de parámetros, se ejecuta periódicamente para leer los valores del encoder en cuadratura, y generar órdenes de movimiento para los motores según esos valores.

CÓDIGO DE EJEMPLO

Como hemos comentado, el código *ESP32_S3_espidf_MCPWM* es prácticamente una copia del código de ejemplo oficial. Simplemente se ha adaptado para PlatformIO, y se han usado los valores de GPIO para las salida PWM y entradas de encoders correspondientes al motor izquierdo de tu robot.

Si ejecutas el código con la placa del ESP32-S3 conectada a la PCB del microbot, el motor izquierdo debería girar a velocidad constante.

Vamos a ver ahora las partes que tiene el código. Empezamos por el principio de *app_main()*:

```
static motor_control_context_t motor_ctrl_ctx = {
    .pcnt_encoder = NULL,
};

ESP_LOGI(TAG, "Create DC motor");
bdc_motor_config_t motor_config = {
    .pwm_freq_hz = BDC_MCPWM_FREQ_HZ,
    .pwma_gpio_num = BDC_MCPWM_GPIO_A,
    .pwmb_gpio_num = BDC_MCPWM_GPIO_B,
};
bdc_motor_mcpwm_config_t mcpwm_config = {
    .group_id = 0,
    .resolution_hz = BDC_MCPWM_TIMER_RESOLUTION_HZ,
};
bdc_motor_handle_t motor = NULL;
ESP_ERROR_CHECK(bdc_motor_new_mcpwm_device(&motor_config, &mcpwm_config,
&motor));
motor_ctrl_ctx.motor = motor;
```

En estas líneas se configura el manejador *motor*, de tipo *bdc_motor_handle_t*, con los valores de frecuencia y resolución del timer que controla el PWM, y los pines de salida PWM que conectarán con el puente en H. El manejador de motor se incluye en una estructura *motor_control_context_t*.

```
ESP_LOGI(TAG, "Init pcnt driver to decode rotary signal");
```

```

pcnt_unit_config_t unit_config = {
    .high_limit = BDC_ENCODER_PCNT_HIGH_LIMIT,
    .low_limit = BDC_ENCODER_PCNT_LOW_LIMIT,
    .flags.accum_count = true, // enable counter accumulation
};
pcnt_unit_handle_t pcnt_unit = NULL;
ESP_ERROR_CHECK(pcnt_new_unit(&unit_config, &pcnt_unit));
pcnt_glitch_filter_config_t filter_config = {
    .max_glitch_ns = 1000,
};
ESP_ERROR_CHECK(pcnt_unit_set_glitch_filter(pcnt_unit, &filter_config));
pcnt_chan_config_t chan_a_config = {
    .edge_gpio_num = BDC_ENCODER_GPIO_A,
    .level_gpio_num = BDC_ENCODER_GPIO_B,
};
pcnt_channel_handle_t pcnt_chan_a = NULL;
ESP_ERROR_CHECK(pcnt_new_channel(pcnt_unit, &chan_a_config,
&pcnt_chan_a));
pcnt_chan_config_t chan_b_config = {
    .edge_gpio_num = BDC_ENCODER_GPIO_B,
    .level_gpio_num = BDC_ENCODER_GPIO_A,
};
pcnt_channel_handle_t pcnt_chan_b = NULL;
ESP_ERROR_CHECK(pcnt_new_channel(pcnt_unit, &chan_b_config,
&pcnt_chan_b));
ESP_ERROR_CHECK(pcnt_channel_set_edge_action(pcnt_chan_a,
PCNT_CHANNEL_EDGE_ACTION_DECREASE, PCNT_CHANNEL_EDGE_ACTION_INCREASE));
ESP_ERROR_CHECK(pcnt_channel_set_level_action(pcnt_chan_a,
PCNT_CHANNEL_LEVEL_ACTION_KEEP, PCNT_CHANNEL_LEVEL_ACTION_INVERSE));
ESP_ERROR_CHECK(pcnt_channel_set_edge_action(pcnt_chan_b,
PCNT_CHANNEL_EDGE_ACTION_INCREASE, PCNT_CHANNEL_EDGE_ACTION_DECREASE));
ESP_ERROR_CHECK(pcnt_channel_set_level_action(pcnt_chan_b,
PCNT_CHANNEL_LEVEL_ACTION_KEEP, PCNT_CHANNEL_LEVEL_ACTION_INVERSE));
ESP_ERROR_CHECK(pcnt_unit_add_watch_point(pcnt_unit,
BDC_ENCODER_PCNT_HIGH_LIMIT));
ESP_ERROR_CHECK(pcnt_unit_add_watch_point(pcnt_unit,
BDC_ENCODER_PCNT_LOW_LIMIT));
ESP_ERROR_CHECK(pcnt_unit_enable(pcnt_unit));
ESP_ERROR_CHECK(pcnt_unit_clear_count(pcnt_unit));
ESP_ERROR_CHECK(pcnt_unit_start(pcnt_unit));
motor_ctrl_ctx.pcnt_encoder = pcnt_unit;

```

En estas líneas se configura y emplea un driver PCNT disponible para nuestro ESP32-S3. Es, básicamente, un contador de pulsos. Es un poco liso, pero este trozo de código básicamente toma las entradas A y B del encoder, e implementa el encoder en

cuadratura, como una unidad PCNT llamada `pcnt_unit`. Esta unidad se añade, junto al manejador del motor, a la estructura `motor_control_context_t`.

```
ESP_LOGI(TAG, "Create PID control block");
pid_ctrl_parameter_t pid_runtime_param = {
    .kp = 0.6,
    .ki = 0.4,
    .kd = 0.2,
    .cal_type = PID_CAL_TYPE_INCREMENTAL,
    .max_output = BDC_MCPWM_DUTY_TICK_MAX - 1,
    .min_output = 0,
    .max_integral = 1000,
    .min_integral = -1000,
};
pid_ctrl_block_handle_t pid_ctrl = NULL;
pid_ctrl_config_t pid_config = {
    .init_param = pid_runtime_param,
};
ESP_ERROR_CHECK(pid_new_control_block(&pid_config, &pid_ctrl));
motor_ctrl_ctx.pid_ctrl = pid_ctrl;

ESP_LOGI(TAG, "Create a timer to do PID calculation periodically");
const esp_timer_create_args_t periodic_timer_args = {
    .callback = pid_loop_cb,
    .arg = &motor_ctrl_ctx,
    .name = "pid_loop"
};
esp_timer_handle_t pid_loop_timer = NULL;
ESP_ERROR_CHECK(esp_timer_create(&periodic_timer_args, &pid_loop_timer));

ESP_LOGI(TAG, "Enable motor");
ESP_ERROR_CHECK(bdc_motor_enable(motor));
ESP_LOGI(TAG, "Forward motor");
ESP_ERROR_CHECK(bdc_motor_forward(motor));

ESP_LOGI(TAG, "Start motor speed loop");
ESP_ERROR_CHECK(esp_timer_start_periodic(pid_loop_timer,
BDC_PID_LOOP_PERIOD_MS * 1000));
```

En esta parte del código, finalmente, se configura el controlador PID, llamado `pid_ctrl`, y se añade a la estructura `motor_control_context_t`, como bloque de control, completándola (la estructura ya tiene el generador de PWM, el lector de encoder, y el bloque de control, las tres cosas que necesitamos para controlar el motor). Después de

eso, se usan las funciones correspondientes de la librería *bdc_motor* para habilitar y poner en marcha el motor. También se pone en marcha un timer periódico que interrumpe cada 10ms (10000 μ s). En ese timer se realizará la lectura del encoder y el establecimiento de la velocidad por parte del controlador PID, tal y como se muestra en este último trozo del código:

```
static void pid_loop_cb(void *args)
{
    static int last_pulse_count = 0;
    motor_control_context_t *ctx = (motor_control_context_t *)args;
    pcnt_unit_handle_t pcnt_unit = ctx->pcnt_encoder;
    pid_ctrl_block_handle_t pid_ctrl = ctx->pid_ctrl;
    bdc_motor_handle_t motor = ctx->motor;

    // get the result from rotary encoder
    int cur_pulse_count = 0;
    pcnt_unit_get_count(pcnt_unit, &cur_pulse_count);
    int real_pulses = cur_pulse_count - last_pulse_count;
    last_pulse_count = cur_pulse_count;
    ctx->report_pulses = real_pulses;

    // calculate the speed error
    float error = BDC_PID_EXPECT_SPEED - real_pulses;
    float new_speed = 0;

    // set the new speed
    pid_compute(pid_ctrl, error, &new_speed);
    bdc_motor_set_speed(motor, (uint32_t)new_speed);
}
```